Assignment 03

Digital RTL Design

Sequential Circuit Design

Marwan Abdelfattah

Contents

1 Question 1 **3**
1.1 Self-Checking Testbench for DFF with Clock Enable and PRE 3
1.2 RTL Design 3
1.3 Testbench 3
1.4 Simulation Waveform 5

2 Question 2 **6**
2.1 Parameterized Asynchronous Flip-Flop Verification 6
2.1.1 Testbench 1: D Flip-Flop Verification 6
2.1.2 Testbench 2: T Flip-Flop Verification 6
2.1.3 Testbench Requirements 6
2.2 Testbench 1: D Flip-Flop 7
2.3 Testbench 2: T Flip-Flop 8
2.4 RTL Design 8

3 Question 3 **10**
3.1 BCD Up Counter (MOD-10 Counter) 10
3.1.1 Inputs 10
3.1.2 Output 10
3.1.3 Functional Requirements 10
3.2 RTL Design 10
3.3 Testbench 12
3.4 Simulation Waveform 12

4 Question 4 **13**
4.1 4-Bit Ripple Counter Using Structural Modeling 13
4.1.1 Inputs 13
4.1.2 Output 13
4.1.3 Functional Requirements 13
4.2 RTL Design 13
4.3 Testbench 14
4.4 Simulation Waveform 14

5 Question 5 **16**
5.1 Parameterized Shift Register 16
5.1.1 Parameters 16
5.1.2 Inputs 16
5.1.3 Outputs 16
5.1.4 Functional Requirements 16
5.2 RTL Design 17
5.3 Testbench 18
5.4 Simulation Waveform 21

6 Question 6 **22**
6.1 Sequential Logic Element (SLE) 22
6.1.1 Functional Requirements 22
6.2 RTL Design 22

6.3 Testbench 23
6.4 Simulation Waveform 24

1 Question 1

1.1 Self-Checking Testbench for DFF with Clock Enable and PRE

Write a **self-checking testbench** for Question 2 in Lab 2, which implements a D-type flip-flop with a clock enable signal and a `PRE` control signal.

The testbench should verify the functionality of all control inputs and automatically check that the output of the design matches the expected result.

1.2 RTL Design

Recall RTL Design Code of DFF From Lab 2

```
module dff (
    input D, E, PRE, CLK,
    output reg Q
);
always @(posedge CLK or negedge PRE) begin
    if(~PRE)begin
        Q <= 1'b1;
    end
    else if (E) begin
        Q <= D;
    end
end
endmodule
endmodule
```

1.3 Testbench

```
module dff_tb ();
reg D, E, PRE, CLK;
wire Q;
reg Q_temp;

dff DUT(D,E,PRE,CLK,Q);
integer i = 0;

initial begin
    CLK = 0;
    forever begin
        #1 CLK = ~CLK;
    end
end

initial begin
    PRE = 0;
    D = 0;
    E = 0;
```

```
20
21      @(negedge CLK);
22
23       PRE = 1;
24
25       for (i=0; i<10; i=i+1) begin
26          D = $random();
27          @(negedge CLK);
28          if(Q != 1'b1)begin
29              $display("Error - DFF Output is incorrect");
30              $stop;
31          end
32       end
33
34       E = 1;
35
36       for (i=0; i<10; i=i+1) begin
37          D = $random();
38          @(negedge CLK);
39          if(Q != D)begin
40              $display("Error - DFF Output is incorrect");
41              $stop;
42          end
43       end
44
45       for(i=0;i<10;i=i+1)begin
46          Q_temp = Q;
47          D = $random();
48          E = $random();
49
50          @(negedge CLK);
51
52          if((E == 1 & Q != D) || (E == 0 && Q != Q_temp))begin
53              $display("Error - DFF Output is incorrect");
54              $stop;
55          end
56       end
57
58       $stop;
59  end
60  endmodule
```

1.4 Simulation Waveform

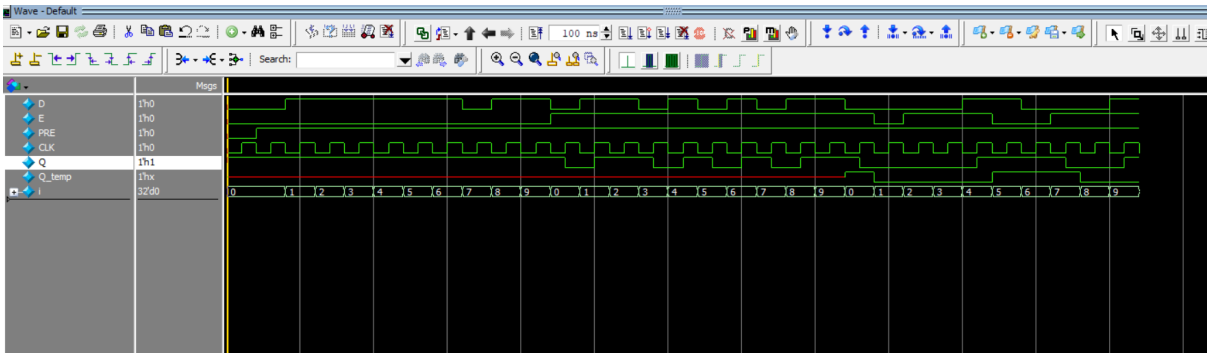


Figure 1: Simulation waveform

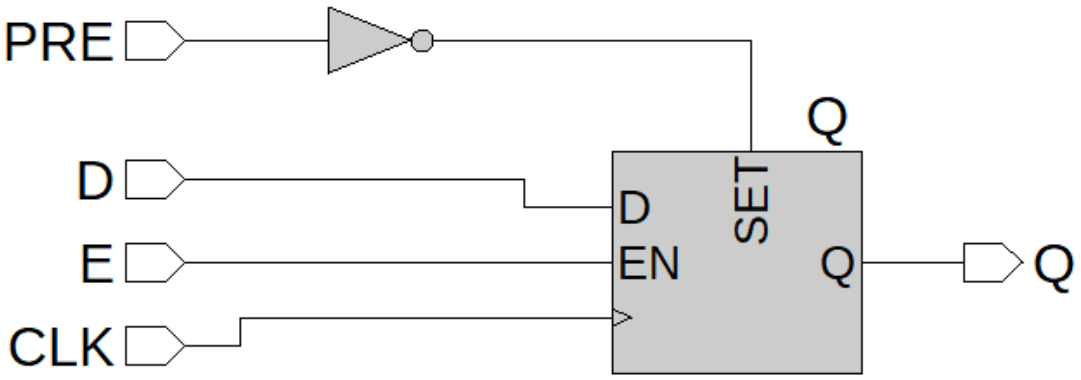


Figure 2: Schematic Diagram

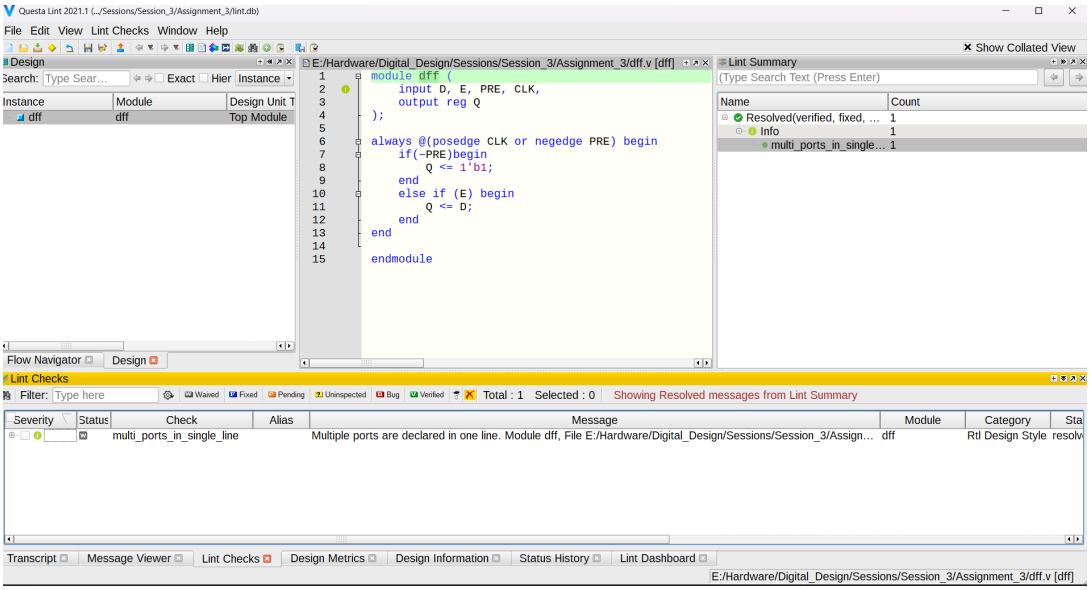


Figure 3: Questa Lint Proof of 0 Errors

2 Question 2

2.1 Parameterized Asynchronous Flip-Flop Verification

Write a **Verilog testbench** for the **parameterized asynchronous Flip-Flop** implemented in **Question 5, Part C of Assignment 2**.

The design supports two operating modes based on the `FF_TYPE` parameter:

- `FF_TYPE = "DFF"`: D Flip-Flop operation.
- `FF_TYPE = "TFF"`: T Flip-Flop operation.

Create **two separate testbenches** to verify the functionality of the design under both parameter configurations.

2.1.1 Testbench 1: D Flip-Flop Verification

Override the design parameter with:

- `FF_TYPE = "DFF"`

The testbench must instantiate the parameterized asynchronous Flip-Flop in DFF mode and verify its functionality.

2.1.2 Testbench 2: T Flip-Flop Verification

Override the design parameter with:

- `FF_TYPE = "TFF"`

The testbench must instantiate the parameterized asynchronous Flip-Flop in TFF mode and verify its functionality.

2.1.3 Testbench Requirements

- Verify the asynchronous reset functionality in both operating modes.
- Apply randomized input stimulus to verify the behavior of the Flip-Flop.
- Verify the DFF operation when `FF_TYPE = "DFF"`.
- Verify the TFF operation when `FF_TYPE = "TFF"`.
- Use self-checking conditions to compare the actual outputs against the expected results.
- Verify the functionality of both configurations using **QuestaSim simulation waveforms**.

2.2 Testbench 1: D Flip-Flop

```
module dff_param_tb ();


reg d, rstn, clk;
wire q_golden, qbar_golden;
wire q, qbar;

//Golden Model Instantiation
D_FF GOLDEN(d,rstn,clk,q_golden,qbar_golden);

//DUT Instantiation
D_FF_param #("DFF") DUT(d,rstn,clk,q,qbar);

initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

integer i;
initial begin
   rstn = 0;
   d = 0;
   @(negedge clk);
   rstn = 1;

   for(i=0;i<20;i=i+1)begin
        d = $random();
        @(negedge clk);

        if(q != q_golden || qbar != qbar_golden)begin
          $display("Error");
          $stop;
        end
   end
   $stop;
end

initial begin
    $monitor("rstn=%d, d=%d, q=%d,q_golden=%d,qbar=%d,qbar_golden=%d",
                rstn, d, q, q_golden, qbar, qbar_golden);
end


endmodule
```

2.3 Testbench 2: T Flip-Flop

```
module tff_param_tb ();

reg d, rstn, clk;
wire q_golden, qbar_golden;
wire q, qbar;

//Golden Model Instantiation
T_FF GOLDEN(d,rstn,clk,q_golden,qbar_golden);

//DUT Instantiation
D_FF_param #("TFF") DUT(d,rstn,clk,q,qbar);

initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

integer i;
initial begin
   rstn = 0;
   d = 0;
   @(negedge clk);
   rstn = 1;

   for(i=0;i<20;i=i+1)begin
        d = $random();
        @(negedge clk);

        if(q != q_golden || qbar != qbar_golden)begin
          $display("Error");
          $stop;
        end
   end
   $stop;
end

initial begin
    $monitor("rstn=%d, d=%d, q=%d,q_golden=%d,qbar=%d,qbar_golden=%d",
                rstn, d, q, q_golden, qbar, qbar_golden);
end


endmodule
```

2.4 RTL Design

Recall RTL Design Code of DFF From Lab 2

```
module D_FF_param (d, rstn, clk, q, qbar);
    parameter FF_TYPE = "DFF";
    input d, rstn, clk;
    output reg q, qbar;

always @(posedge clk or negedge rstn) begin
    if (~rstn) begin
        q <= 1'b0;
        qbar <= 1'b1;

    end else begin
        if (FF_TYPE == "DFF") begin
            q <= d;
            qbar <= ~d;
        end
        else if (FF_TYPE == "TFF") begin
            if(d) begin
                q <= ~q;
                qbar <= ~qbar;
            end
        end
    end

end
endmodule
```

3 Question 3

3.1 BCD Up Counter (MOD-10 Counter)

Implement a **BCD up counter (MOD-10 counter)** using Verilog HDL. The counter consists of **10 states**, counting from 0 to 9, and is used to divide the input clock frequency by 10.

3.1.1 Inputs

- `Clk`: Clock input.
- `Rst`: Asynchronous active-high reset input.

3.1.2 Output

- `Clk_div10_out`: Clock output with a frequency equal to the input clock frequency divided by 10.

3.1.3 Functional Requirements

- The internal counter must count sequentially from 0 to 9 and then return to 0.
- The counter must support an asynchronous active-high reset that initializes the counter and output to zero.
- The output `Clk_div10_out` must toggle based on the internal counter value to achieve a clock frequency division by 10.

The output frequency is given by:

$$f_{\text{out}} = \frac{f_{\text{Clk}}}{10}$$

Hint: Toggle the output every five input clock cycles to generate an output clock with a period equal to ten input clock cycles.

3.2 RTL Design

```
1  module bcd_counter (
2      input clk, rst,
3      output reg clock_div10_out
4  );
5
6  reg [3:0] count;
7
8  always @(posedge clk or posedge rst) begin
9      if(rst) begin
10
11       count <= 0;
12       // clock_div10_out <= 0;
13
```

```
14      end else if (count == 4'b1001) begin
15
16        count <= 0;
17        // clock_div10_out <= ~clock_div10_out;
18
19      end else begin
20        count <= count + 1;
21      end
22  end
23
24  always @(posedge clk or posedge rst) begin
25    if (rst) begin
26      clock_div10_out <= 0;
27    end else begin
28      if (count < 5) begin
29        clock_div10_out <= 0;
30      end else begin
31        clock_div10_out <= 1;
32      end
33    end
34
35  end
36
37  endmodule
```

3.3 Testbench

```
module bcd_counter_tb ();
    reg clk, rst;
    wire clock_div10_out;
    bcd_counter DUT(clk, rst, clock_div10_out);
    initial begin
        clk = 0;
        forever begin
           #1 clk = ~clk;
        end
    end
    initial begin
        rst = 1;
        @(negedge clk);
        rst = 0;
        repeat (100) @(negedge clk);
        $stop;
    end

endmodule
```

3.4 Simulation Waveform

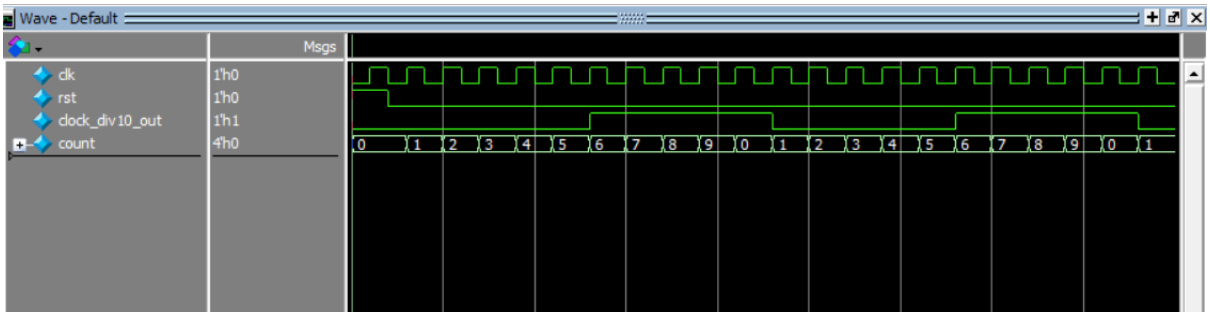


Figure 4: Simulation waveform

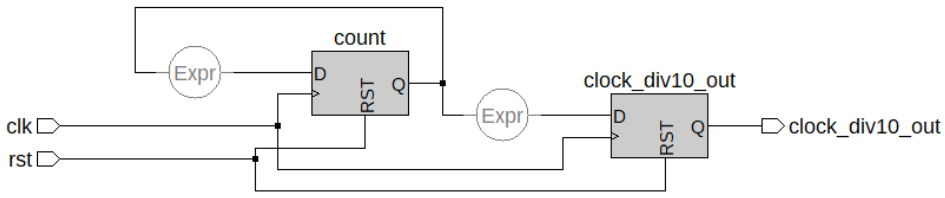


Figure 5: Schematic Diagram

4 Question 4

4.1 4-Bit Ripple Counter Using Structural Modeling

Implement a **4-bit Ripple Counter** using **structural modeling** in Verilog HDL.

The design must instantiate the **D Flip-Flop (DFF)** implemented in **Question 5, Part B of Assignment 2**, where the output is taken from the `qn` signal, as illustrated in the provided circuit diagram.

4.1.1 Inputs

- `clk`: Clock input.
- `rstn`: Asynchronous active-low reset input.

4.1.2 Output

- `out[3:0]`: 4-bit ripple counter output.

4.1.3 Functional Requirements

- Implement the counter using **structural modeling** by instantiating the previously designed D Flip-Flop module.
- Connect the flip-flops according to the provided ripple counter circuit diagram.
- Use the `qn` output of the D Flip-Flops as specified in the circuit.
- The counter must support an asynchronous active-low reset through `rstn`.
- The 4-bit counter must generate the counting sequence according to the provided circuit.

4.2 RTL Design

```
module ripple_counter (
    input clk,rstn,
    output [3:0] out
);
wire q0,qn0 , q1,qn1, q2,qn2 ,q3,qn3;

D_FF FF1 (.d(qn0), .rstn(rstn), .clk(clk),.q(q0),.qbar(qn0)) ;
D_FF FF2 (.d(qn1), .rstn(rstn), .clk(q0),.q(q1),.qbar(qn1));
D_FF FF3 (.d(qn2), .rstn(rstn), .clk(q1),.q(q2),.qbar(qn2));
D_FF FF4 (.d(qn3), .rstn(rstn), .clk(q2),.q(q3),.qbar(qn3));

assign out = {qn3,qn2,qn1,qn0};

endmodule
```

4.3 Testbench

```
module ripple_tb ();
reg clk, rstn;
wire [3:0] out;
ripple_counter DUT(clk, rstn, out);

initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

initial begin
    rstn = 0;
    @(negedge clk);
    rstn = 1;
    repeat(100) @(negedge clk);
    $stop;
end

initial begin
    $monitor("rstn=%b, clk=%b, out= %b", rstn, clk, out);
end
endmodule
```

4.4 Simulation Waveform

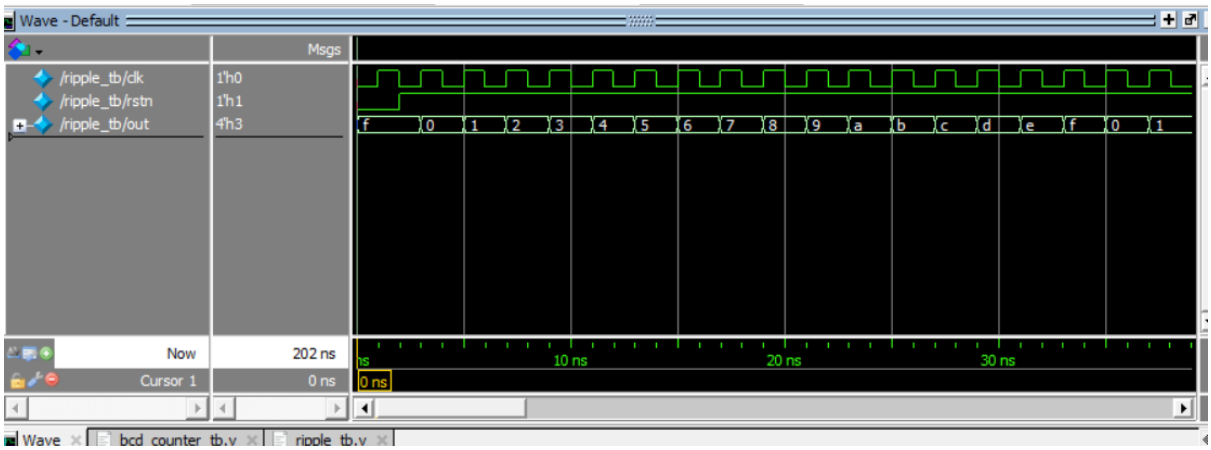


Figure 6: Simulation waveform

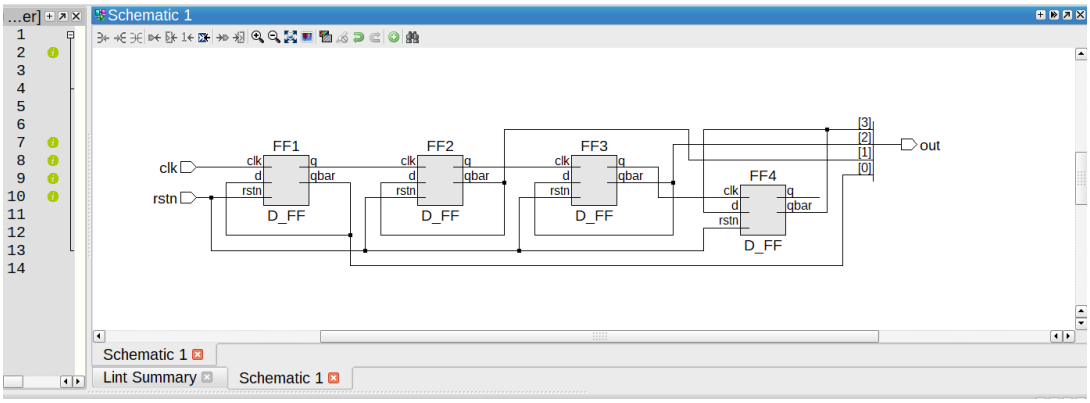


Figure 7: Schematic Diagram

5 Question 5

5.1 Parameterized Shift Register

Implement a **parameterized shift register** using **Verilog HDL**. The design must support asynchronous and synchronous control operations, parallel loading, and configurable shifting direction and register width.

5.1.1 Parameters

- `LOAD_AVALUE`: Specifies the asynchronous load value. Default value: `1`.
- `LOAD_SVALUE`: Specifies the synchronous load value. Default value: `1`.
- `SHIFT_DIRECTION`: Determines the shifting direction, either `"LEFT"` or `"RIGHT"`.
- `SHIFT_WIDTH`: Determines the width of the shift register. Default value: `8`.

5.1.2 Inputs

- `clk`: Clock input.
- `aclr`: Asynchronous clear input.
- `aset`: Asynchronous set input.
- `sclr`: Synchronous clear input.
- `sset`: Synchronous set input.
- `enable`: Enables synchronous operation.
- `load`: Parallel load control signal.
- `data`: Parallel input data to be loaded into the shift register.
- `shiftin`: Serial input for shifting.

5.1.3 Outputs

- `q`: Parallel output representing the current contents of the shift register.
- `shiftout`: Serial output representing the bit shifted out of the register.

5.1.4 Functional Requirements

- The `enable` signal controls the synchronous operation of the shift register.
- The `enable` signal has priority over the synchronous control signals `sclr` and `sset`.
- The synchronous control signals `sclr` and `sset` have priority over the `load` signal.
- The design must support asynchronous clear and asynchronous set operations.

- The register must support parallel loading and shifting according to the selected direction.
- The `shiftout` signal must represent the actual bit removed from the register during shifting, rather than always representing the most significant bit.

5.2 RTL Design

```
module param_shift_reg #(
    parameter  LOAD_AVALUE = 1,
    parameter  SHIFT_DIRECTION = "LEFT",
    parameter  LOAD_SVALUE = 1,
    parameter  SHIFT_WIDTH = 8

) (

    input sclr, sset, shiftin, load, clk, enable, aclr, aset,
    input [SHIFT_WIDTH-1:0] data,
    output reg shiftout,
    output reg [SHIFT_WIDTH-1:0] q
);

always @(posedge clk or posedge aclr or posedge aset) begin
    if(aclr)begin
      q <= {SHIFT_WIDTH{1'b0}};

    end else if (aset) begin
      q <= LOAD_AVALUE;

    end else if (enable) begin

        if(sclr)begin
            q <= {SHIFT_WIDTH{1'b0}};

        end else if (sset) begin
            q <= LOAD_SVALUE;

        end else if(load)begin

            q <= data;

        end else begin
           if (SHIFT_DIRECTION == "LEFT") begin
                {shiftout, q}<= {q, shiftin};
            end else
                {q, shiftout} <= {shiftin, q};
        end
    end
end

endmodule
```

5.3 Testbench

```
module param_shift_reg_tb ();

 parameter  LOAD_AVALUE = 2;
 parameter  SHIFT_DIRECTION = "LEFT";
 parameter  LOAD_SVALUE = 4;
 parameter  SHIFT_WIDTH = 8;

reg sclr, sset, shiftin, load, clk, enable, aclr, aset;
reg [SHIFT_WIDTH-1:0] data;
wire shiftout;
wire [SHIFT_WIDTH-1:0] q;

param_shift_reg #(LOAD_AVALUE, SHIFT_DIRECTION, LOAD_SVALUE,SHIFT_WIDTH) DUT
    (.*);

initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

integer i;

// 2.1 ACLR Functionality CHECK
initial begin
    aclr = 1;
    aset = 1;
    for(i=0;i<10;i=i+1) begin
      sclr = $random;
      sset = $random;
      shiftin = $random;
      load = $random;
      enable = $random;
      data = $random;

      @(negedge clk);

      if(q != {SHIFT_DIRECTION{1'b0}}) begin
        $display("ERROR - ACLR FAILED");
        $stop;
      end
    end

// 2.2 ASET Functionality CHECK
    aclr = 0;
    for(i=0;i<10;i=i+1) begin
      sclr = $random;
      sset = $random;
```

```
      shiftin = $random;
      load = $random;
      enable = $random;
      data = $random;

      @(negedge clk);

      if(q != LOAD_AVALUE) begin
        $display("ERROR - ASET FAILED");
        $stop;
      end
    end

// 2.3 SCLR Functionality CHECK

    aset = 0;
    sclr = 1;
    sset = 1;
    for(i=0;i<10;i=i+1) begin
      shiftin = $random;
      load = $random;
      enable = $random;
      data = $random;

      @(negedge clk);

      if(q != 1'b0 && enable == 1) begin
        $display("ERROR - SCLR FAILED");
        $stop;
      end
    end

// 2.4 SSET Functionality CHECK
    sclr = 0;
    for(i=0;i<10;i=i+1) begin
      shiftin = $random;
      load = $random;
      enable = $random;
      data = $random;

      @(negedge clk);

      if(q != LOAD_SVALUE && enable == 1) begin
        $display("ERROR - SSET FAILED");
        $stop;
      end
    end

// 2.5 load Functionality CHECK
    sset = 0;
    load = 1;
```

```
100     for(i=0;i<10;i=i+1) begin
101       shiftin = $random;
102 
103       enable = $random;
104       data = $random;
105 
106       @(negedge clk);
107 
108       if(q != data && enable == 1) begin
109         $display("ERROR - LOAD FAILED");
110         $stop;
111       end
112     end
113 
114     // 2.6  shift Functionality CHECK
115     load = 0;
116     enable = 1;
117     for(i=0;i<10;i=i+1) begin
118       shiftin = $random;
119       data = $random;
120 
121       @(negedge clk);
122     end
123 
124 
125 
126     enable = 0;
127     for(i=0;i<10;i=i+1) begin
128       shiftin = $random;
129       data = $random;
130 
131       @(negedge clk);
132     end
133 
134 
135 
136     for(i=0;i<10;i=i+1) begin
137       enable = $random;
138       shiftin = $random;
139       data = $random;
140       @(negedge clk);
141     end
142 
143   $stop;
144 end
```

5.4 Simulation Waveform

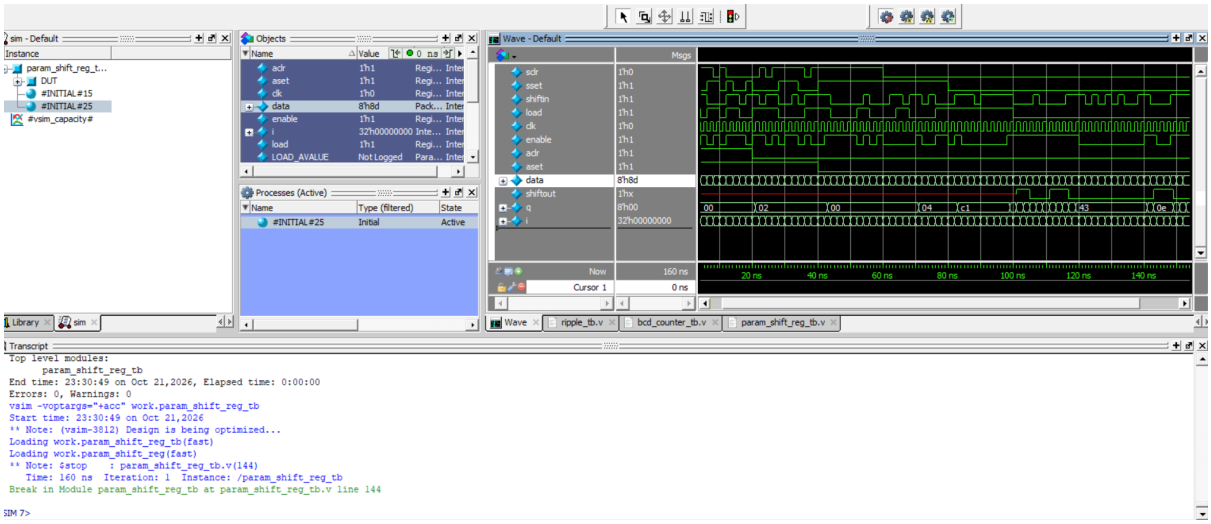


Figure 8: Simulation waveform

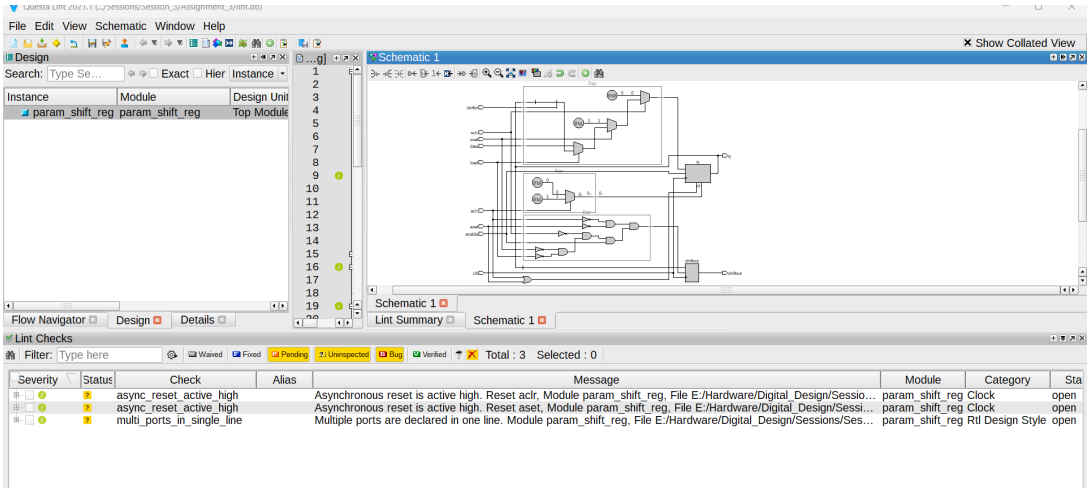


Figure 9: Schematic Diagram

6 Question 6

6.1 Sequential Logic Element (SLE)

Implement a **Sequential Logic Element (SLE)** using **Verilog HDL**. The design must operate as either a **flip-flop** or a **latch**, depending on the `LAT` control signal, as specified in the provided truth table.

6.1.1 Functional Requirements

- Implement the SLE according to the provided truth table.
- The `LAT` signal determines whether the circuit operates as a flip-flop or a latch.
- The design must support the `ALn` control signal and the remaining control inputs according to the specified truth table.
- Verify the functionality of each control input in both operating modes.

6.2 RTL Design

```
module SLE (
    input clk,
    input D, EN, ALn, ADn, SLn, SD, LAT,
    output Q
);
reg Q_ff, Q_latch;
always @(posedge clk or negedge ALn) begin
    if (!ALn) begin
        Q_ff <= !ADn;
    end else if (EN) begin
        if (!SLn) begin
            Q_ff <= SD;
        end else
            Q_ff <= D;
    end
end
always @(*) begin
    if (!ALn) begin
        Q_latch <= !ADn;
    end else if (EN) begin
        if (!SLn) begin
            Q_latch <= SD;
        end else
            Q_latch <= D;
    end
end
assign Q = (LAT)? Q_latch: Q_ff;
endmodule
```

6.3 Testbench

```
module SLE_tb ();
    reg clk;
    reg D, EN, ALn, ADn, SLn, SD, LAT;
    wire Q;

    SLE  DUT(.*);

    initial begin
        clk = 0;
        forever begin
            #1 clk = ~clk;
        end
    end

    initial begin
        ALn = 0;
        ADn = 0;
        LAT = 0;
        EN = 0;
        D = 0;
        SLn = 0;
        SD = 0;

        @(negedge clk);

        ALn = 1;

        repeat (100)  begin
          EN=  $random();
          D=   $random();
          SLn= $random();
          SD = $random();
          ADn = $random();
          @(negedge clk);
        end

        LAT = 1;

        repeat (100)  begin
          EN=  $random();
          D=   $random();
          SLn= $random();
          SD = $random();
          ADn = $random();
          @(negedge clk);
        end

    end

```

```
50 initial begin
51     $monitor("ALn=%b, ADn=%b, LAT=%b, EN=%b, SLn=%b,SD=%b, D=%b, Q=%b",
52                   ALn, ADn, LAT, EN, SLn, SD, D, Q);
53
54 end
55 endmodule
```

6.4 Simulation Waveform

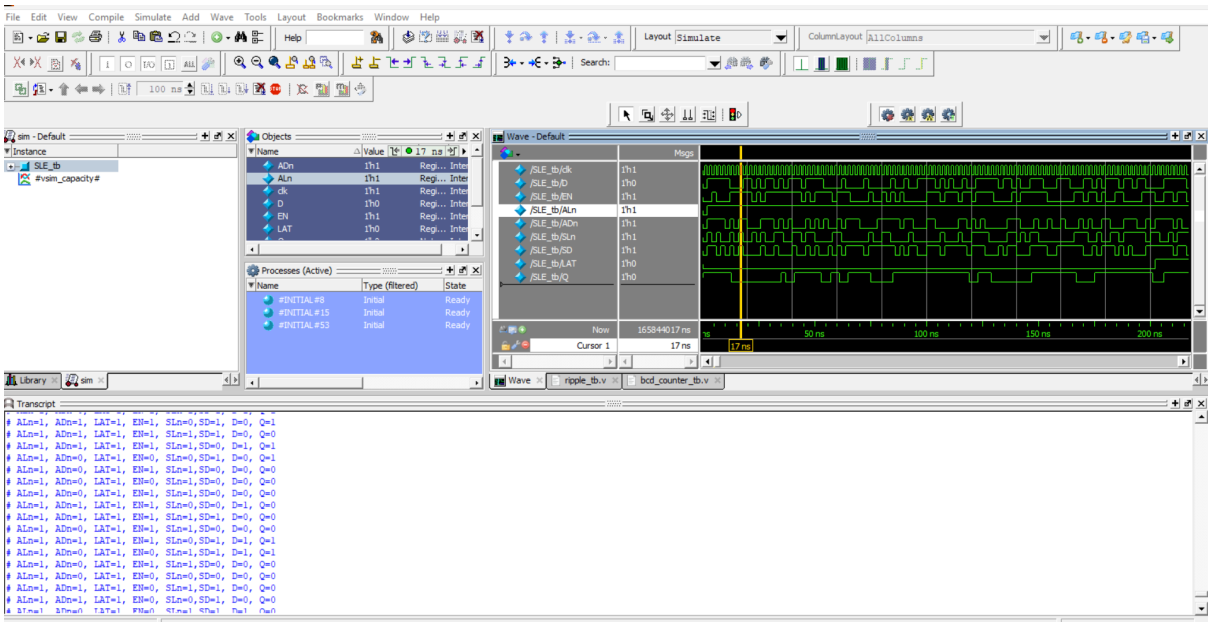


Figure 10: Simulation waveform

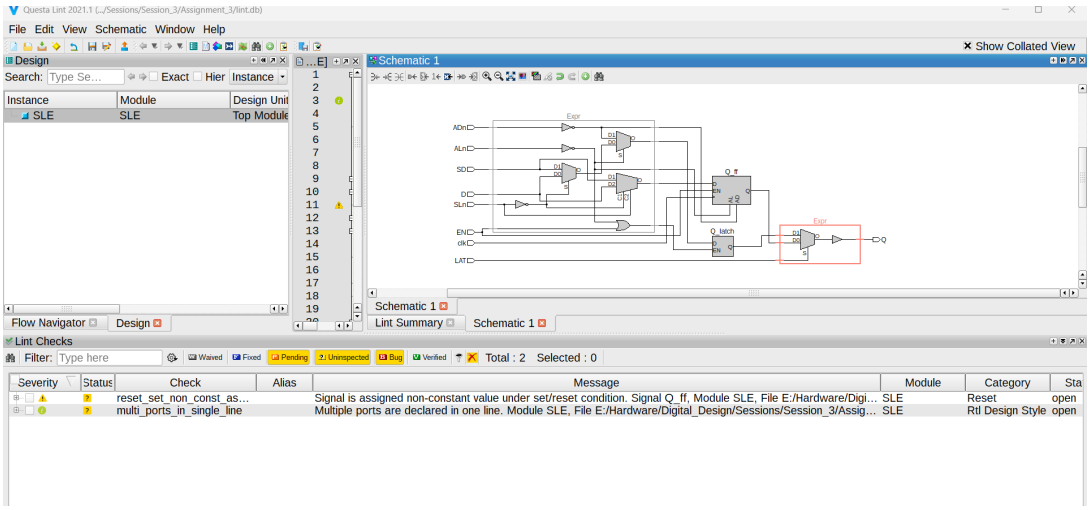


Figure 11: Schematic Diagram